ROADS Prototype: Pipeline & Multi-Loan Architecture

Nov 9, 2026 · @Ben

Where the prototype stands today

The hard part of multi-loan support is already built: every loan is one self-contained JSON document, persisted per `loan_id` in Postgres, and every detail-page feature reads and writes only through that document. What is missing is everything above one loan: a list, a create path, a persisted stage, and routing that lives outside `/loans/:id`. The table is what I found in the uploaded code; the rest of this doc builds on it rather than replacing it.

Layer	Today, as read from the code	Consequence for the pipeline
Stack	React 18 + Vite 7, wouter 3 routing, TanStack Query 5, Tailwind + shadcn; Express 5; raw `pg` SQL; Drizzle schema present but unused by the loan API; vitest suite	No new framework needed; add routes, one table query, one page
Routes (`client/src/App.tsx`)	`/` redirects to `/loans/1`; six routes `/loans/:id`, `/origination`, `/decisioning`, `/closing-booking`, `/documents`, `/history`; NotFound fallback	Add `/pipeline`; the six loan routes stay byte-identical
Loan identity	One seed loan `id: "1"` with 3 borrowers in `MemStorage` (`server/storage.ts`); `load()` in `server/loan-application.ts` bootstraps its Postgres document from that seed on first read	Seed loan 1 stays as-is; new loans never touch `MemStorage`
Persistence	Table `prototype_loan_applications(loan_id text PK, document jsonb)`; `document` = `{ loan, borrowers, mode, generation, revision, sections, changeLog, businessRules }`	This document is the loan package; the pipeline is an index over these rows

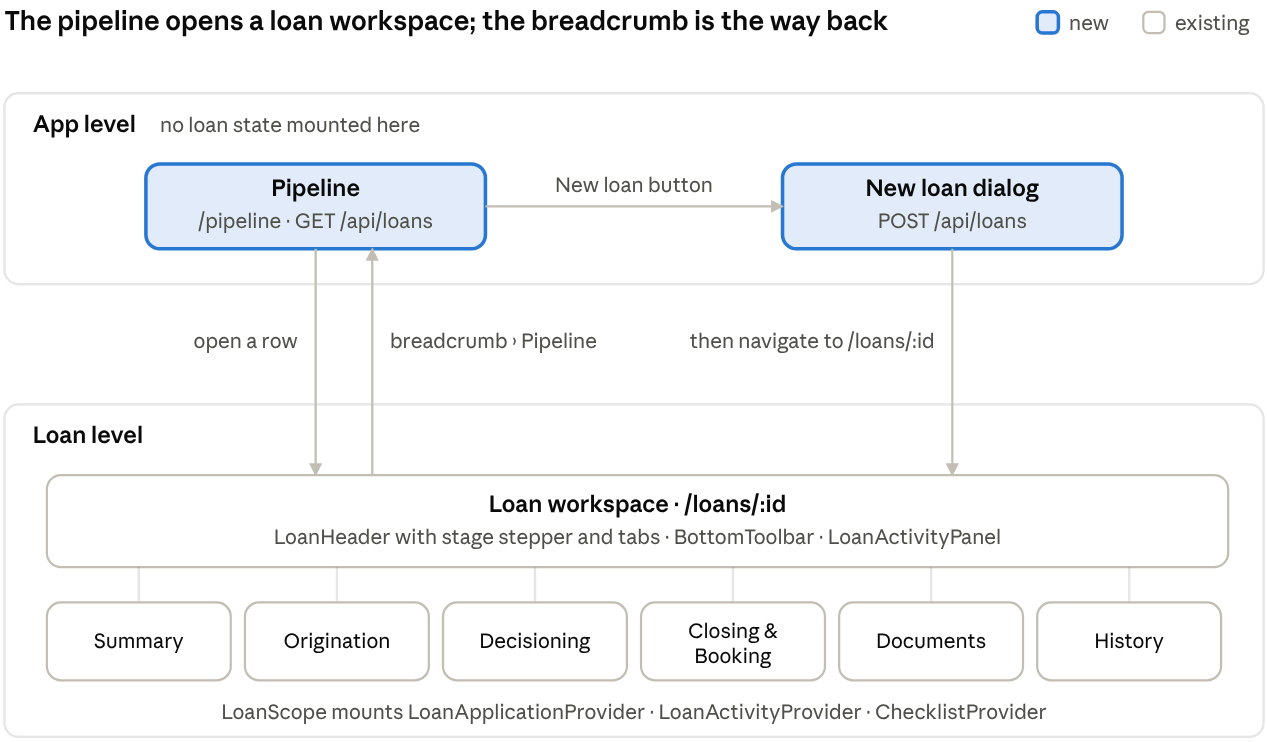
Layer	Today, as read from the code	Consequence for the pipeline
Write path	`PUT /api/loans/:id/application` writes one `section` at a time with compare-and-swap on `generation` + `revision`; the server recomputes `businessRules` and appends `changeLog`	Reused unchanged for every loan; stage becomes one more CAS write
Client data access	`applicationStore` singleton (`client/src/lib/loan-application-store.ts`) holds one active loan per tab; `LoanApplicationProvider` reads `:id` from the URL, loads, and gates children until loaded; `useLoanDraft`, `useLoanState` and `loanStorage` are the only read/write surface	Works for N loans already; the provider is a pass-through when the URL has no `/loans/:id`, so a pipeline page renders outside it today
Loan-scoped side stores	Checklist, activity, closing package, disclosures, subject property are keyed `prefix:loanId` and routed through `loanStorage` into `sections["storage:…"]`	Already isolated per loan (your `loan-demo-ownership` memory note anticipated this)
Stage	`loan.stage` exists in the schema and seed, but `StageStepper` in `LoanHeader.tsx` is local demo state and never persists	Stage must become persisted data before a pipeline can group by it
Page shell	Each of the six pages repeats `useLoanQuery`, loading/error states, `LoanHeader` + content + `BottomToolbar` + `LoanActivityPanel`, with a hard-coded fallback to loan `"1"` when `params.id` is missing	Extract one `LoanWorkspace` layout and a `useLoanId()` hook; delete the `"1"` fallbacks

Layer	Today, as read from the code	Consequence for the pipeline
Create / reset	`POST /api/loans/:id/application/demo` with `reset` or `blend` already constructs a blank or Blend-seeded document (zeroed loan template, `mode: "empty"` or `"blend"`, fresh borrowers, filtered sections) but only ever replaces loan 1	The loan factory already exists inside that handler; extract it and point it at a new id
Dead ends	A second `GET /api/loans/:id` in `server/routes.ts` is shadowed by the one registered first in `loan-application.ts`; the Drizzle `loans`, `borrowers` and `users` tables are never queried	Remove the shadowed route; leave the Drizzle tables alone (prototype scope)

Two details matter most for what follows. `mode: "existing"` means a legacy seed loan whose defaults live in client constants such as `RICHARD_DATA`; a loan created from the pipeline must never carry that mode, or it renders Richard Jamerson's data. And the Blend fixture (`client/src/lib/blend-demo-fixture.ts`) builds its sections from component constants on the client, so Blend-seeded creation should keep flowing through the existing client → `/demo` path rather than be re-implemented on the server.

Target information architecture

Two levels, not one: an **app level** (pipeline and, later, any other list or dashboard page) and a **loan level** (the existing workspace at `/loans/:id` with its six tabs). The pipeline is the new root; a loan is opened from it and returns to it through the breadcrumb that already reads "Pipeline › Loan: 123456789" in `LoanHeader.tsx`.



page hierarchy · app level above loan level, two ways in, one way back

The six tabs and everything under them are untouched; the two new pieces sit above them and only ever hand off a loan id.

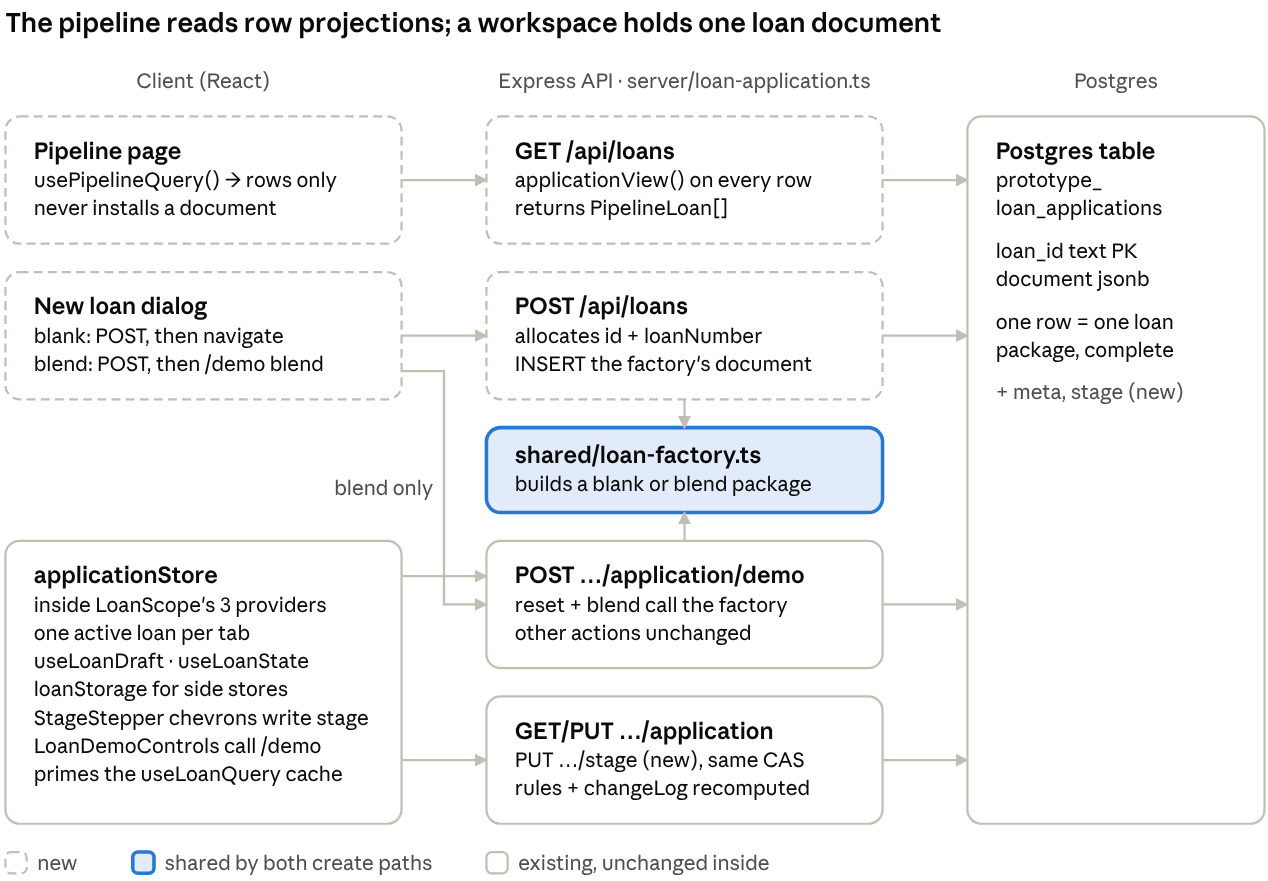
Route	Page	Scope	Status
`/`	Redirect to `/pipeline`	app	changes (today redirects to `/loans/1`)
`/pipeline`	Pipeline list, New loan dialog	app	new
`/loans/:id`	Summary	loan	existing
`/loans/:id/origination`	Origination (side nav: Borrower Information, Loan Details, Collateral, Product & Pricing, Fees)	loan	existing
`/loans/:id/decisioning`	Decisioning	loan	existing

Route	Page	Scope	Status
`/loans/:id/closing-booking`	Closing & Booking	loan	existing
`/loans/:id/documents`	Documents	loan	existing
`/loans/:id/history`	History	loan	existing
anything else	Not Found	app	existing

There is deliberately no `/loans/new` route. Creating a loan is a `POST` that returns an id, followed by navigation to `/loans/:id`; the existing provider then loads it like any other loan. Future app-level pages (a dashboard, reports, settings) hang off the same app shell beside `/pipeline` without touching the loan level.

The pattern: loan as aggregate root, pipeline as an index

One loan is one `LoanApplication` document, and that document is the entire package: header fields, borrowers, every saved section, the checklist, activity, documents state, change log and business-rule results. The pipeline never opens those packages; it reads a small projection of each one. Creating a loan means inserting a new package built by a factory, then navigating into it. Nothing inside the package changes to support many of them.



component map · client, API routes and the one Postgres table

Everything new (dashed) sits beside the existing loan path, not inside it; the factory is the one piece both the demo actions and `POST /api/loans` share.

Five rules fall out of this, and they are the rules to hand to Replit:

1. **The document is the only unit of loan state.** Every feature on the detail page already reads and writes through `applicationStore` by section key within the active loan. Keep it that way: no new per-loan tables, no second store, no cross-loan reads from inside a loan page.
2. **The pipeline reads projections, not documents.** `GET /api/loans` returns one `PipelineLoan` row per document, computed on the server by the existing `applicationView()` plus stage and timestamps. The client never holds two full loan documents at once, so the single-active-loan `applicationStore` stays correct.
3. **Creation is a factory, and the factory already exists.** The code that builds a blank or Blend document inside the `reset` / `blend` demo action moves to `shared/loan-factory.ts`. `POST /api/loans` and the demo actions both call it; the only difference is whether the target `loan_id` is new or current.

4. **Stage is data, not UI state.** `loan.stage` is persisted in the document, written through the same compare-and-swap path as any section, and read by the stepper, the pipeline chips and the factory default. The 8 stage names move from `LoanHeader.tsx` to `shared/loan-stages.ts` so all three agree.
5. **Two provider levels.** App-level providers (QueryClient, GitHub sync, Toaster, Tooltip) wrap everything. Loan-level providers (`LoanApplicationProvider` , `LoanActivityProvider` , `ChecklistProvider`) mount only inside the `/loans/:id` branch. The pipeline page lives outside them and has no way to touch loan state by accident.

What this pattern deliberately avoids: normalizing the document into the unused Drizzle `loans` / `borrowers` tables (a rewrite with no prototype payoff), a client-side list that fetches every document (breaks the singleton store and gets slow fast), and a "template loan" row that new loans clone (the factory plus the existing `mode: "empty"` blank-field behaviour already give you an empty package).

Data model and persistence

Keep the one table. `prototype_loan_applications` already holds one row per loan, so the pipeline is `SELECT loan_id, document FROM prototype_loan_applications` projected on the server. The only schema-level change is inside the JSON: a `meta` block for timestamps and origin, and `loan.stage` becoming authoritative. No Drizzle migration, no new tables, no `db:push` (your SETUP.md flags that command as needing separate authorization).

```
// shared/loan-application.ts — additions only; nothing existing changes
shape
export interface LoanApplication {
  loan: Loan;                        // loan.stage is now persisted truth, one
of LOAN_STAGES
  borrowers: Borrower[];
  mode: "existing" | "empty" | "blend";   // "existing" is reserved for seed
loan 1
  generation: number;
  revision: number;
  sections: Record<string, unknown>;
  changeLog?: LoanChange[];
  businessRules?: LoanBusinessRules;
  meta?: {                           // NEW — load() backfills it for loan 1
the same way it backfills businessRules
```

```
    createdAt: string;              // ISO
    updatedAt: string;              // ISO; every UPDATE sets it with one
more jsonb_set
    source: "seed" | "blank" | "blend";
  };
}

// shared/loan-stages.ts — moved out of LoanHeader.tsx so stepper, pipeline
and factory agree
export const LOAN_STAGES = ["Customer Application", "Pre-Approval", "Loan
Setup", "Decisioning",
  "Loan Finalization", "Closing", "Booking", "Loan Certification"] as const;
export type LoanStage = typeof LOAN_STAGES[number];

// shared/pipeline.ts — what the list page receives; computed from
applicationView(document)
export interface PipelineLoan {
  id: string;
  loanNumber: string;
  primaryBorrowerName: string;
  loanType: string;
  stage: LoanStage;
  amountRequested: number | null;   // summary.amountRequested
  product: string | null;           // summary.product
  mode: LoanApplication["mode"];
  revision: number;
  createdAt: string;
  updatedAt: string;
}
```

Identity: new loans get `id = crypto.randomUUID()` (opaque, never edited, safe in URLs) and a 9-digit `loanNumber` allocated as max existing + 1, so the first created loan after the seed is `123456790`. Seed loan `"1"` keeps its id so existing links, tests and the `/loans/1` habit keep working. Borrower ids follow the existing `${loan.id}-primary` / `-secondary` convention from the demo action.

Route	Purpose	Status	Notes
`GET /api/loans`	Pipeline rows (`PipelineLoan[]`), newest `updatedAt` first	new	Projects each document server-side; `Cache-Control: no-store` like the loan routes
`POST /api/loans`	Create a blank loan, return `{ id, loanNumber }`	new	Body `{ source: "blank" }`; calls the factory; `INSERT … ON CONFLICT DO NOTHING`
`PUT /api/loans/:id/stage`	Persist `loan.stage`	new	Body `{ stage, generation, revision }`; same CAS and 409 message pattern as `PUT …/application`
`GET /api/loans/:id`	Header + summary view	existing	Unchanged; delete the shadowed duplicate in `server/routes.ts`
`GET` / `PUT /api/loans/:id/application`	Load and section writes	existing	Unchanged except `meta.updatedAt` is set on write
`POST /api/loans/:id/application/demo`	Reset, Blend import, TRID, checklist	existing	`reset` and `blend` now call the shared factory; everything else unchanged
`DELETE /api/loans/:id`	Remove a loan from the pipeline	later	Refuse for seed loan `"1"`; prototype can hard-delete the row

Blend-seeded creation is two calls, not one: `POST /api/loans` makes the blank package, then the client sends the existing `/demo` action `blend` with `blendDemoSections(newId, borrowerCount)`. That reuses the fixture that lives in client constants and the server code that already filters and installs those sections, at the cost of one extra round trip nobody will notice.

Routing and client state

Move the three loan providers from `AppContent` into a `LoanScope` branch under `/loans/:id`, give the pipeline its own branch beside it, and replace every hard-coded `"1"` fallback with a `useLoanId()` hook. The six page components keep their content and lose their repeated shell. This is the target `App.tsx`:

```
// client/src/App.tsx (target)
function Router() {
  return (
    <Switch>
      <Route path="/"><Redirect to="/pipeline" /></Route>
      <Route path="/pipeline" component={PipelinePage} />
      <Route path="/loans/:id" nest>
        <LoanScope>                     {/* reads :id, provides
LoanIdContext, mounts the three loan providers */}
          <LoanWorkspace>               {/* LoanHeader + BottomToolbar +
LoanActivityPanel + loading/error, once */}
            <Switch>
              <Route path="/origination" component={LoanOriginationPage} />
              <Route path="/decisioning" component={LoanDecisioningPage} />
              <Route path="/closing-booking" component=
{LoanClosingBookingPage} />
              <Route path="/documents" component={LoanDocumentsPage} />
              <Route path="/history" component={LoanHistoryPage} />
              <Route path="/" component={LoanSummaryPage} />
            </Switch>
          </LoanWorkspace>
        </LoanScope>
      </Route>
      <Route component={NotFound} />
    </Switch>
  );
}
```

Piece	File	What it does
`LoanIdContext` + `useLoanId()`	`client/src/contexts/loan-id-context.tsx` (new)	`LoanScope` reads `useParams().id` once and provides it; pages, `LoanHeader` and the contexts call `useLoanId()` instead of re-parsing `window.location` or falling back to `"1"`. Throws if used outside a loan, which is the point
`LoanScope`	`client/src/components/loan/LoanScope.tsx` (new)	Wraps `LoanApplicationProvider`, `LoanActivityProvider`, `ChecklistProvider`; nothing else. `LoanApplicationProvider` already reloads when `:id` changes and gates children until the document is installed, so pipeline → loan → pipeline → other loan needs no new logic
`LoanWorkspace`	`client/src/components/loan/LoanWorkspace.tsx` (new)	The shell every page repeats today: `useLoanQuery(useLoanId())`, the loading and error states, `LoanHeader` with the active tab derived from the route, `BottomToolbar`, `LoanActivityPanel`. Pages render only their content column
`AppHeader`	`client/src/components/AppHeader.tsx` (new)	The breadcrumb bar extracted from `LoanHeader.tsx`'s `BreadcrumbBar`, minus the loan crumb: menu icon, "Pipeline" as a `Link`, Viewing avatar, more-options menu. The GitHub sync panel moves here because sync is app-level; the demo controls stay in the loan header's menu because they act on `store.document`
`usePipelineQuery()`	`client/src/hooks/use-pipeline-query.ts` (new)	`useQuery({ queryKey: ["/api/loans"], refetchOnMount: "always" })`. The default `staleTime: Infinity` in `queryClient.ts` would otherwise hide loans a collaborator created; invalidate after create and after a stage write
`useLeaveLoan()`	`client/src/hooks/use-leave-loan.ts` (new)	`await applicationStore.flush()` then navigate; if `store.dirty`, confirm first. Needed because `applicationStore.write` rejects a queued save once `document.loan.id` changes, so the breadcrumb must drain the queue before leaving

Two things stay exactly as they are. The `applicationStore` singleton remains one-active-loan-per-tab; the pipeline never installs a document, so it cannot collide with it. And `useLoanQuery(["/api/loans", id])` keeps being primed by `applicationStore.install()`, so the header still updates from saves without a refetch.

The only route-shaped risk: wouter's `nest` makes the inner paths relative, so `TAB_ROUTES` in `LoanHeader.tsx` must keep building absolute hrefs (`/loans/${id}/origination`) and `loan-origination.tsx` must keep reading `?section=` from `useSearch()`, which is

unaffected by nesting.

Creating a loan from the pipeline: the factory

The last 40 lines of the `/demo` handler in `server/loan-application.ts` (from `const seed = await storage.getLoan(...)` down to `next.businessRules = evaluateLoanRules(next)`) are already a loan factory. They zero a loan template, set `mode`, mint `${id}-primary` / `${id}-secondary` borrowers, filter the allowed section keys and recompute rules. Extract that into a pure function; the demo action and `POST /api/loans` both call it.

```
// shared/loan-factory.ts
import type { LoanApplication } from "./loan-application";
import { evaluateLoanRules } from "./business-rules";

export const BLANK_LOAN_TEMPLATE = {           // replaces copying seed loan
1 and zeroing it
  loanType: "", stage: "Customer Application", primaryBorrowerName: "",
product: "",
  applicationType: "", cifNumber: "", pweCbScore: 0, ruleOfXx: null,
  ecoaDaysRemaining: 0, tridDaysRemaining: 0,
  displayAmount: "0", amountRequested: "0", financedFees: "0",
taxesInsurance: "0",
  finalRate: "0", principalInterest: "0", totalMonthlyPayment: "0", piti:
"0", dti: "0",
  lcv: "0", totalCv: "0", cashReserves: "0", totalLoanAmount: "0",
cashToClose: "0", apr: "0",
} as const;

export const IMPORTABLE_SECTIONS = ["borrower-primary", "borrower-secondary",
"loan-details", "income",
  "current-assets", "noncurrent-assets", "current-liabilities", "noncurrent-
liabilities", "subject-property", "fees"];

export interface NewLoanInput {
  id: string;
  loanNumber: string;
  source: "blank" | "blend";
  borrowerCount: 1 | 2;
  sections?: Record<string, unknown>;   // blend only; client-built via
blendDemoSections(id, count)
```

```
  generation?: number;                    // demo reset passes current+1;
creation passes 0
  revision?: number;
  now?: string;
}

export function newLoanApplication(input: NewLoanInput): LoanApplication {
  const blend = input.source === "blend";
  const loan = { ...BLANK_LOAN_TEMPLATE, id: input.id, loanNumber:
input.loanNumber,
    loanType: blend ? "Purchase" : "",
    applicationType: blend && input.borrowerCount === 2 ? "Joint Credit
Application" : "",
    amountRequested: blend ? "145000.00" : "0", primaryBorrowerName: blend ?
"Richard Jamerson" : "" };
  loan.displayAmount = loan.amountRequested; loan.totalLoanAmount =
loan.amountRequested;
  const borrowers = ["primary", "secondary"].slice(0, blend ?
input.borrowerCount : 2).map((role, i) => ({
    id: `${loan.id}-${role}`, loanId: loan.id, isPrimary: i === 0,
    name: blend ? `${i === 0 ? "Richard" : "Jane"} Jamerson` : "",
    email: blend ? `${i === 0 ? "richard" : "jane"}.jamerson@example.com` :
"",
    phone: blend ? `(502) 555-${i === 0 ? "0142" : "0188"}` : "" }));
  const sections = blend ? Object.fromEntries(Object.entries(input.sections
?? {})
    .filter(([key]) => IMPORTABLE_SECTIONS.includes(key))) : {};
  if (blend && input.borrowerCount === 1) { delete sections["borrower-
secondary"];
    if (Array.isArray(sections.income)) sections.income =
sections.income.slice(0, 1); }
  const now = input.now ?? new Date().toISOString();
  const next: LoanApplication = { loan, borrowers, mode: blend ? "blend" :
"empty", changeLog: [],
    generation: input.generation ?? 0, revision: input.revision ?? 0,
sections,
    meta: { createdAt: now, updatedAt: now, source: input.source } };
  next.businessRules = evaluateLoanRules(next);
  return next;
}
```

The demo action keeps one behaviour the factory must not absorb: when loan 1 is reset, it carries the *shape* of the previous checklist statuses forward as `not-started`. A brand-new loan has no `storage:` keys, and `loadTasks()` in `checklist-context.tsx` already defaults every task to `not-started` whenever `mode !== "existing"`, so creation needs nothing extra.

What the New loan dialog does, in order:

1. User picks a source (Blank application or Blend import) and, for Blend, 1 or 2 borrowers — the same two choices `LoanDemoControls.tsx` offers today.
2. `POST /api/loans { source: "blank" }` → server allocates `id` and `loanNumber`, runs the factory with `source: "blank"`, inserts, returns `{ id, loanNumber }`.
3. Blend only: `applicationRequest(id, "/demo", { action: "blend", borrowerCount, generation: 0, revision: 0, confirmed: true, sections: blendDemoSections(id, borrowerCount) })` — the exact call `LoanDemoControls` already makes, with the new id.
4. `queryClient.invalidateQueries({ queryKey: ["/api/loans"] })`, then navigate: Blank lands on `/loans/${id}/origination?section=Borrower%20Information` so the first thing on screen is an editable form; Blend lands on `/loans/${id}`.

If step 3 fails, the blank loan still exists in the pipeline with `source: "blank"`; show the error in the dialog and leave the row — the user can open it or delete it, and nothing half-written is left in a single document.

`loanNumber` allocation on the server: `SELECT COALESCE(MAX((document->'loan'->>'loanNumber')::bigint), 123456789) + 1`. Two people creating in the same second could collide on the number (never on the id); for a prototype, accept that or retry once on a duplicate.

Pipeline page specification

A single table of every loan, newest activity first, with stage filter chips, a search box and one primary action, **New loan**. Row click opens `/loans/:id`. It is built from `usePipelineQuery()` and `PipelineLoan` rows only; it never loads a loan document.

Column	Source field	Display	Sort
Loan #	`loanNumber`	text, `body-200-strong`; this is the row's link	asc / desc

Column	Source field	Display	Sort
Primary borrower	`primaryBorrowerName`	text; "—" when empty (blank loans)	asc / desc
Loan type	`loanType`	text; "—" when empty	asc / desc
Stage	`stage`	chip, same colours as the completed / active chevrons in `StageStepper`	by `LOAN_STAGES` order
Amount requested	`amountRequested`	`formatCurrency` from `loan-display.ts`; "—" when null	numeric
Product	`product`	text; "—" when null	asc / desc
Last updated	`updatedAt`	relative ("2 min ago") with full timestamp on hover	default, desc

Controls above the table:

- **Stage chips**: one per `LOAN_STAGES` entry plus "All"; multi-select; counts in each chip come from the loaded rows, no extra request.
- **Search**: matches `loanNumber` and `primaryBorrowerName`, case-insensitive, client-side.
- **New loan**: opens the dialog from the previous section. Disabled while a create is in flight.
- **Row menu** (later): Open, Delete (hidden for seed loan `"1"`).

States: a loading skeleton with the same column widths; an error state with a Retry button that refetches; an empty state ("No loans match" when filtered, "No loans yet — create the first one" when the table is truly empty, which cannot happen while seed loan 1 exists).

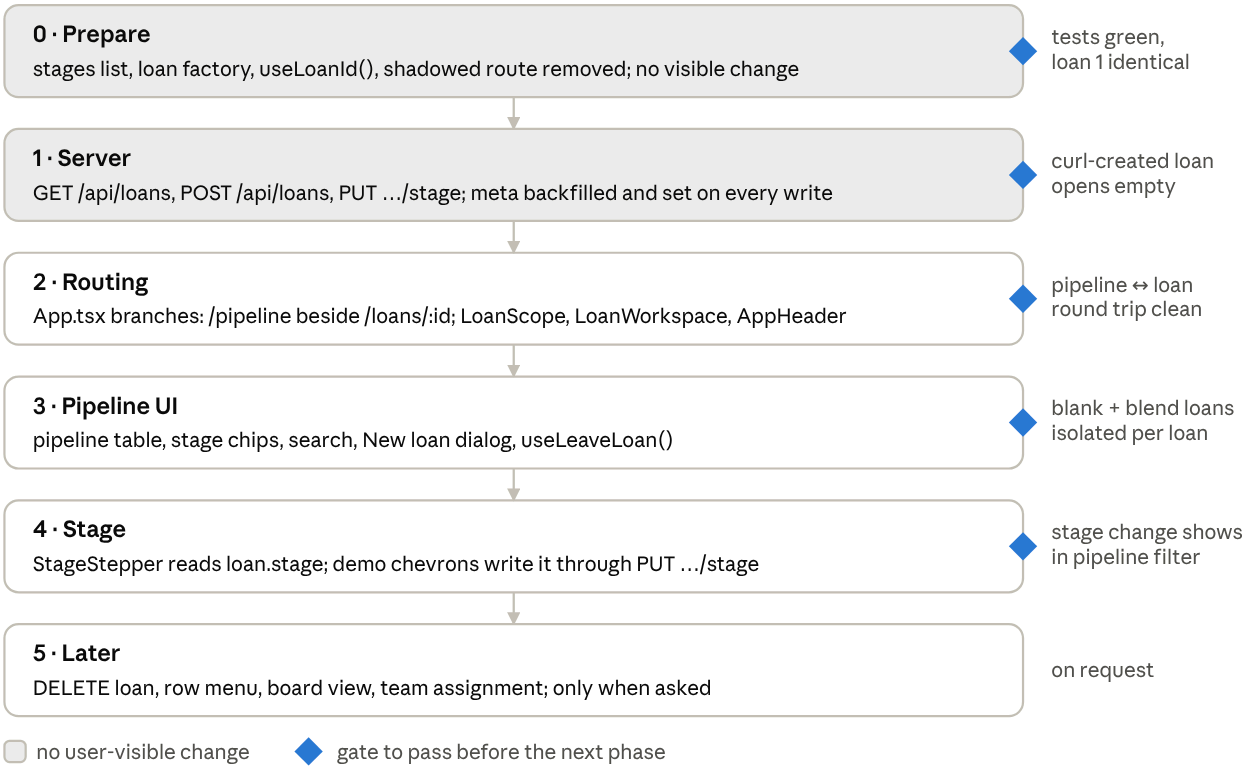
Layout and design-system rules, carried over from `replit.md`: `AppHeader` on top (breadcrumb shows just "Pipeline"), content in the same `--roads-bg-page` surface, `headline-100` page title, table typography from the existing `body-100` / `caption-100` classes, all colours and spacing from `--roads-*` tokens. No new tokens without asking first (your token approval rule), and no arbitrary pixel values beyond the structural widths Figma already allows. If the Figma file has a Pipeline frame, point Replit at it; if not, say so explicitly so it does not invent one.

Out of scope for the first version, by design: a kanban board view grouped by stage (the stage chips give 80% of that), assignment to a loan team member (the "Loan Team" avatars in the header are static today), and server-side paging (a prototype with tens of loans does not need it).

Migration plan

Six phases, each small enough for one Replit session and each gated on loan 1 behaving exactly as it does today. Phases 0 and 1 change no visible behaviour, which is where the risk of breaking the detail page is lowest and the payoff of catching a regression is highest.

Two invisible phases first; every gate checks that loan 1 still works



roadmap · 6 phases, 5 gates

Each phase is one Replit session; the gate is what you check in the browser before pasting the next prompt.

Phase	Touches	Gate to pass before the next phase
0 · Prepare	`shared/loan-stages.ts`, `shared/loan-factory.ts`, `client/src/contexts/loan-id-context.tsx`; delete the shadowed `GET /api/loans/:id` in `routes.ts`; replace `params.id || "1"` in six pages and the `window.location` parse in `LoanHeader.tsx` with `useLoanId()`; demo `reset` / `blend` call the factory	`npm run check` clean; full vitest suite green; `/loans/1` pixel-identical; Reset and Blend demo actions produce the same document as before (compare JSON)
1 · Server	`GET /api/loans`, `POST /api/loans`, `PUT /api/loans/:id/stage`; `meta` backfill in `load()`; `meta.updatedAt` on every UPDATE	`curl POST /api/loans` returns an id; `/loans/<id>` opens empty with all tasks not-started; `GET /api/loans` lists both loans
2 · Routing	`App.tsx` restructure, `LoanScope`, `LoanWorkspace`, `AppHeader`, breadcrumb `Link`, GitHub sync moved to `AppHeader`; placeholder `/pipeline` page	`/` lands on the placeholder; pipeline → loan 1 → pipeline → loan `<id>` → loan 1 with no stale header, checklist or activity; sync panel reachable from both levels
3 · Pipeline UI	`pages/pipeline.tsx`, `use-pipeline-query.ts`, stage chips, search, table, states; `NewLoanDialog.tsx`; `use-leave-loan.ts`	Create blank and Blend loans from the dialog; edit each; History, checklist and activity stay isolated per loan; leaving a loan mid-save does not lose the save
4 · Stage	`StageStepper` reads `loan.stage` and the demo chevrons write it through `PUT …/stage`; pipeline chip updates after navigating back	Change stage on a loan, return to the pipeline, filter by that stage
5 · Later	`DELETE /api/loans/:id` + row menu; board view; team assignment	Only when asked

The change-review habit already in `replit.md` carries over: Replit should ask before any ambiguous UX choice, must not touch `design-tokens.css` or `design-tokens.ts` without approval, and must not push to GitHub unless told. Phase 0 is also the moment to tell

Replit that `mode: "existing"` is reserved for loan 1 and that `MemStorage` is not to grow new loans.

Prompts to feed Replit

One prompt per phase, written to be pasted as-is. Each is self-contained because Replit will not have this document unless you paste it; the first prompt carries the rules every later one assumes. Run them in order and do not start the next until the previous gate passes. If you want Replit to see the full reasoning as well, export this doc as Markdown and add it to the repo as `docs/pipeline-architecture.md` before Prompt 0, then tell Replit to read it.

Prompt 0 — Prepare (no visible change)

```
We are expanding ROADS to support many loans and a pipeline page above them.
This first task changes no user-visible behaviour; it prepares the code.
Rules for this and every later task: (1) loan 1's pages must look and behave
exactly as today; (2) every loan is one `LoanApplication` document in
`prototype_loan_applications` — do not add tables, do not use the Drizzle
`loans`/`borrowers` tables, do not run db:push; (3) `mode: "existing"` is
reserved for seed loan 1 and new loans must never carry it; (4) `MemStorage`
keeps only seed loan 1; (5) do not touch design-tokens.css or design-
tokens.ts; (6) do not push to GitHub; (7) ask before guessing at any UX
choice.

Do the following, then run `npm run check` and the vitest suite and report
results:
1. Create `shared/loan-stages.ts` exporting `LOAN_STAGES` (the 8 labels now
hard-coded in `client/src/components/loan/LoanHeader.tsx`) and `LoanStage`.
Make `LoanHeader.tsx` import it.
2. Create `shared/loan-factory.ts` exporting `BLANK_LOAN_TEMPLATE`,
`IMPORTABLE_SECTIONS` and `newLoanApplication(input)`, extracted from the
`reset`/`blend` branch of the `/demo` handler in `server/loan-
application.ts`. Add `meta: { createdAt, updatedAt, source }` to the
`LoanApplication` type in `shared/loan-application.ts` as an optional field.
Make the demo handler call the factory and prove the resulting document is
unchanged (write a vitest comparing old vs new output for both actions and
both borrower counts).
3. Create `client/src/contexts/loan-id-context.tsx` with `LoanIdProvider` and
`useLoanId()`. For now, provide it from `LoanApplicationProvider` using the
id it already parses. Replace every `params.id || "1"` in
`client/src/pages/*.tsx` and the `window.location.pathname` parse in
`LoanHeader.tsx` with `useLoanId()`.
4. Delete the duplicate `app.get("/api/loans/:id", …)` in `server/routes.ts`;
the one in `server/loan-application.ts` is registered first and already wins.
```

Prompt 1 — Server endpoints

```
Same rules as before. Add three endpoints to `server/loan-application.ts`, matching the style, CAS pattern and 409/503 error messages already used there:
- `GET /api/loans`: select every row of `prototype_loan_applications`, run each document through `load()`'s backfill logic (so `meta` and `businessRules` exist) and `applicationView()`, and return `PipelineLoan[]` sorted by `meta.updatedAt` descending. Define `PipelineLoan` in `shared/pipeline.ts` with: id, loanNumber, primaryBorrowerName, loanType, stage, amountRequested (summary.amountRequested), product (summary.product), mode, revision, createdAt, updatedAt. Set `Cache-Control: no-store`.
- `POST /api/loans` with body `{ source: "blank" }`: allocate `id = crypto.randomUUID()` and `loanNumber = COALESCE(MAX((document->'loan'->>'loanNumber')::bigint), 123456789) + 1` as a string; build with `newLoanApplication({ id, loanNumber, source: "blank", borrowerCount: 2 })`; `INSERT … ON CONFLICT DO NOTHING`; return `{ id, loanNumber }` with 201.
- `PUT /api/loans/:id/stage` with body `{ stage, generation, revision }`: validate `stage` against `LOAN_STAGES`, CAS-update `document.loan.stage` and `document.meta.updatedAt`, bump revision, return the document.
Also: have `load()` backfill `meta` for loan 1 (`source: "seed"`) exactly the way it backfills `businessRules`, and add a `jsonb_set` for `meta.updatedAt` to every existing UPDATE statement in that file.
Verify with curl: create a loan, open `/loans/<id>` in the browser and confirm it renders empty with every checklist task not-started, then confirm `GET /api/loans` lists both loans.
```

Prompt 2 — Routing and shells

```
Same rules as before. Restructure routing so loan-scoped providers live only under `/loans/:id`:
1. In `client/src/App.tsx`, replace the Router with: `/` → Redirect to `/pipeline`; `/pipeline` → a placeholder `PipelinePage` that renders `AppHeader` and the text "Pipeline"; `<Route path="/loans/:id" nest>` wrapping `<LoanScope><LoanWorkspace><Switch>` with the six existing pages at relative paths (`/origination`, `/decisioning`, `/closing-booking`, `/documents`, `/history`, `/` for Summary); then NotFound.
2. `LoanScope` (new, `client/src/components/loan/LoanScope.tsx`) reads `useParams().id`, provides `LoanIdProvider`, and mounts `LoanApplicationProvider`, `LoanActivityProvider`, `ChecklistProvider` — moved out of `AppContent`. QueryClient, Tooltip, Toaster and GitHub sync stay app-level.
3. `LoanWorkspace` (new) owns what the six pages repeat today: `useLoanQuery(useLoanId())`, the loading and error states, `LoanHeader` with the active tab derived from the current route, `BottomToolbar`, `LoanActivityPanel`. Reduce each page to its content column. Keep every `data-testid` so existing tests pass.
4. `AppHeader` (new, `client/src/components/AppHeader.tsx`): extract `BreadcrumbBar` from `LoanHeader.tsx` into a shared bar that takes crumbs as props; the loan header passes `["Pipeline", "Loan: <number>"]` with "Pipeline" as a wouter `Link` to `/pipeline`; the app header passes `["Pipeline"]`. Move the GitHub sync panel trigger into this shared bar so it works on both levels; keep the Loan Demo Controls tab visible only when a loan is active.
5. `TAB_ROUTES` in `LoanHeader.tsx` must keep producing absolute paths; `?section=` on Origination must keep working.
Gate: pipeline → loan 1 → pipeline → the loan created in the previous task → loan 1, with the header, checklist and activity panel always showing the current loan's data; vitest green.
```

Prompt 3 — Pipeline page and New loan dialog

```
Same rules as before. Build the real `/pipeline` page and creation flow:
1. `client/src/hooks/use-pipeline-query.ts`: `useQuery<PipelineLoan[]>({
queryKey: ["/api/loans"], refetchOnMount: "always" })`.
2. `client/src/pages/pipeline.tsx`: `AppHeader`, `headline-100` title
"Pipeline", a "New loan" primary button, stage filter chips (one per
`LOAN_STAGES` plus All, multi-select, counts from loaded rows), a search box
matching loanNumber and primaryBorrowerName, and a table with columns Loan #,
Primary borrower, Loan type, Stage (chip using the StageStepper colours),
Amount requested (`formatCurrency`), Product, Last updated (relative, full
timestamp on hover; default sort desc). Row click navigates to `/loans/:id`.
Add loading skeleton, error + Retry, and empty states. Use only `--roads-*`
tokens and existing typography classes; if the Figma file has no Pipeline
frame, follow the Summary page's spacing and say so.
3. `client/src/components/NewLoanDialog.tsx`: source radio (Blank application
/ Blend import) and, for Blend, the same 1-or-2 borrower choice as
`LoanDemoControls.tsx`. On confirm: `POST /api/loans { source: "blank" }`; if
Blend, call `applicationRequest(id, "/demo", { method: "POST", body: {
action: "blend", borrowerCount, generation: 0, revision: 0, confirmed: true,
sections: blendDemoSections(id, borrowerCount) } })`; then invalidate
`["/api/loans"]` and navigate — Blank to `/loans/${id}/origination?
section=Borrower%20Information`, Blend to `/loans/${id}`. Show server errors
inside the dialog; if the Blend step fails, keep the blank loan and say so.
4. `client/src/hooks/use-leave-loan.ts`: `await applicationStore.flush()`
before navigating to `/pipeline`; if `applicationStore.dirty`,
`window.confirm` first. Use it for the Pipeline breadcrumb link.
Gate: create one blank and one Blend loan; edit Borrower Information in each;
confirm History, checklist and activity never show another loan's data; save
a field and immediately click Pipeline — the save must land.
```

Prompt 4 — Persisted stage

```
Same rules as before. Make loan stage real data. `StageStepper` in
`LoanHeader.tsx` currently keeps the stage in local state with `initialStep`
2 or 0. Change it to read `applicationStore.document.loan.stage` (index in
`LOAN_STAGES`) and make the first/last chevron demo controls call `PUT
/api/loans/:id/stage` with the neighbouring stage and the current
`generation`/`revision`, then `applicationStore.install()` the returned
document. Keep the chevrons visually unchanged. New loans start at "Customer
Application" via the factory; loan 1 keeps "Loan Setup" from its seed. The
pipeline's Stage column and chips should reflect a change after navigating
back. Update `replit.md` (Pages, Data Model, API Routes) to describe the
pipeline, the new endpoints and that stage is now persisted.
```

Prompt 5 — Later, only when wanted

```
Add `DELETE /api/loans/:id` (refuse for loan "1"), a row menu on the pipeline
with Open and Delete behind a confirm, and invalidate the pipeline query
after deleting. Do not add a board view or team assignment unless asked.
```